CODEMAX

System design, LLM comparison, advanced RAG evaluation, and reflection

Executive summary

CodeMax is a Python command-line coding agent. It accepts a natural-language task, asks a selected language model what to do, invokes built-in or Model Context Protocol (MCP) tools, feeds results back into the conversation, and repeats until the model returns a final answer or a 15-iteration safety limit is reached. The interactive REPL streams responses and offers confirm-before-execute (default) and auto-execute modes.

Key design decisions

Decision	Rationale and implementation
Provider interface	A shared LangChain-backed **LLMProvider** supports Groq (cloud) and Ollama (local), keeping model selection outside the agent loop.
Agent/tool boundary	The agent uses a reason-act-observe loop with a max-iteration guard, bounded retries, and tool schemas. File writes, edits, and shell execution require confirmation unless the user opts into auto mode.
MCP integration	A stdio MCP client discovers tools at startup from a project-scoped filesystem server, Context7, and a custom RAG server. This lets the same agent use local tools and external services.
Advanced RAG	A persistent Chroma index over RAGAs documentation uses local sentence-transformer embeddings. A cross-encoder reranks the 20 vector candidates before returning the requested top-k passages.

Scope and evidence note

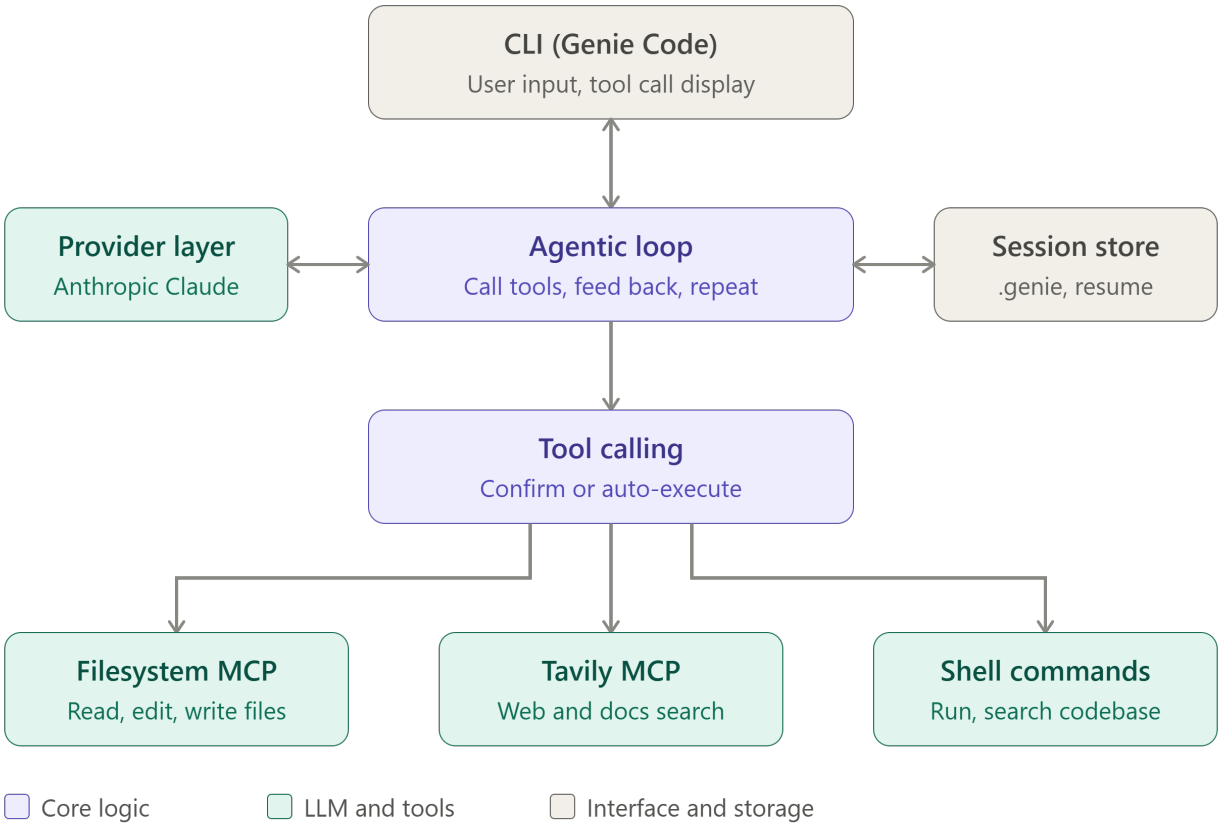
This report reflects the checked-in specification, implementation, demo notes, and a fresh run of the repository RAG evaluation. A controlled, logged run of the exact same coding prompt on two models is not present in the repository. The model section reports the available qualitative observations and explicitly marks that benchmark gap; it does not invent timings, scores, or success rates.

System highlights

Python | LangChain | Groq + Ollama | Rich / prompt_toolkit | MCP | ChromaDB | sentence-transformers

Original planning-phase architecture

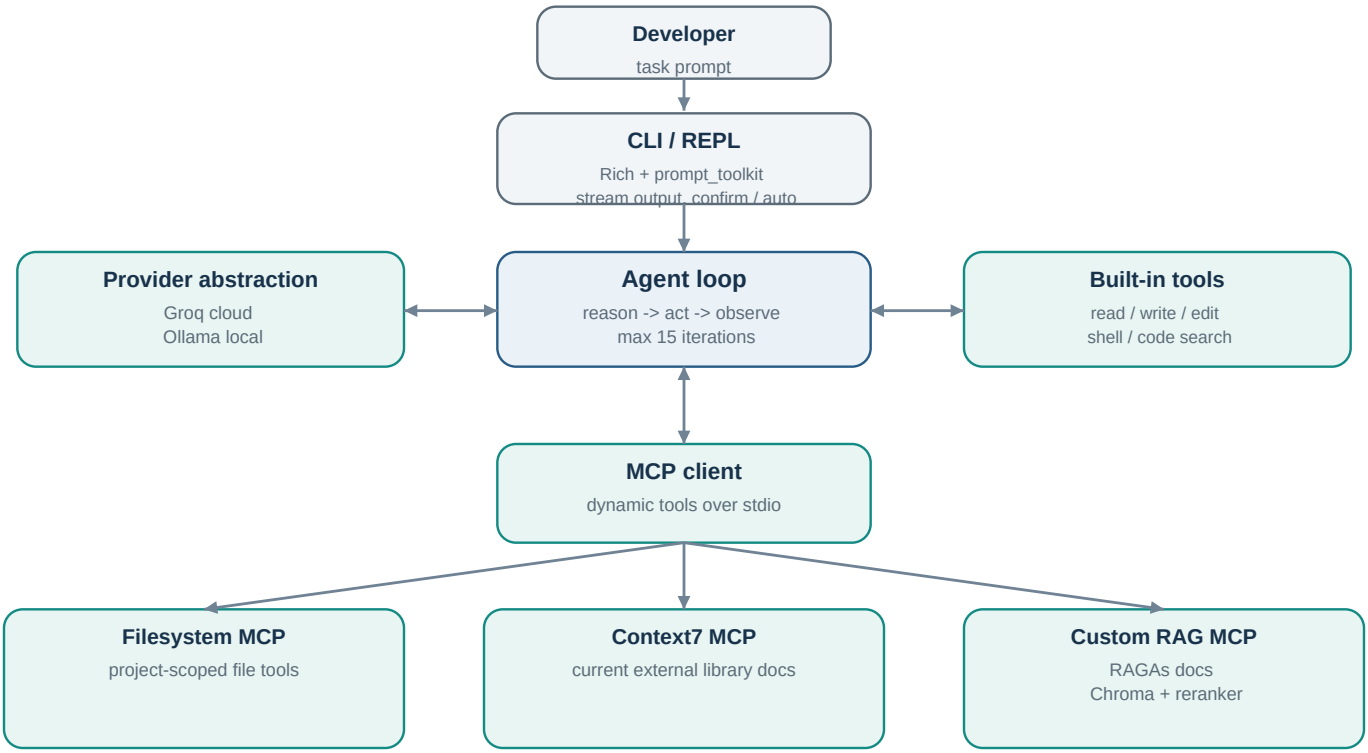
The image below is the original diagram committed with the project. It is preserved as-is; its labels describe an earlier Genie Code concept rather than the final CodeMax implementation.



Reading the original: it sketches a CLI, provider, agent loop, tool calling, three tool destinations, and a session store. Its provider is Anthropic Claude, its external MCP is Tavily, and it includes resumable session storage. These were planning-era choices, not the final configuration.

Updated architecture: implemented CodeMax

This diagram reflects the checked-in package layout and demo configuration. Tool results return through the MCP client or built-in tool wrapper to the agent loop; the selected provider supplies the next response.



Local RAG index: RAGAs Markdown -> heading-aware chunks -> embeddings -> persisted Chroma -> top-20 retrieval -> cross-encoder reranking -> top-k passages

Planning diagram	Implemented system	Why it changed
Genie Code CLI	CodeMax REPL using Rich and prompt_toolkit	The project scope and final branding settled on CodeMax.
Anthropic Claude	Groq cloud or Ollama local, behind a provider interface	The final project specification selected the two providers and enabled a common agent loop.
Tavily MCP	Context7 MCP for current library documentation	The team selected a developer-documentation lookup server instead of a web-search server.
Session store (.genie, resume)	No session persistence / resume component	Session persistence was marked out of scope; the implemented architecture does not include a session store.
Filesystem, web/docs, shell grouped as direct destinations	Built-in tools plus an MCP client dynamically loading filesystem, Context7, and custom RAG tools	The implementation separates built-in tools from MCP transport and adds the custom RAG service.

RAG data path: the ingestion command downloads/reads RAGAs Markdown, splits it by Markdown heading (with bounded overlapping windows for long sections), embeds chunks locally, and persists them in Chroma. At query time the server retrieves up to 20 vector candidates, scores query-passage pairs with the cross-encoder, and returns top-k passages with source metadata.

LLM comparison on coding-agent tool use

The project exposes a Groq cloud model (default: **openai/gpt-oss-120b**) and a local Ollama model (default: **qwen2.5-coder:1.5b**). The committed demo notes record meaningful differences in tool calling under the same agent/tool interface.

Model	Recorded observation	Implication for the coding task
Groq: openai/gpt-oss-120b	The demo notes report occasional invalid tool-call JSON, especially on long file contents. The agent retries provider errors with a corrective instruction; rate-limit handling is also implemented.	It can participate in the structured tool-call loop, but malformed calls and service limits can interrupt a long run. The retry path is useful but is not a guarantee of recovery.
Ollama: qwen2.5-coder:1.5b	The demo notes report tool calls emitted as plain text rather than structured calls.	In the observed setup, it could not drive the autonomous tool loop, so a multi-step task requiring file edits and command execution could not be completed through the agent.

Benchmark limitation: The repository contains no run log showing both models receive the identical coding prompt, task setup, and scoring procedure. It also contains no paired success, test-pass, step-count, token, or elapsed-time measurements. Therefore this is a qualitative comparison of recorded tool-use behavior, not a controlled same-task performance result. No numerical model winner can be claimed from this evidence.

Insights from the comparison

For an autonomous coding agent, structured tool-call reliability is a prerequisite: a model that discusses an edit in ordinary text cannot substitute for a model that returns callable, schema-conforming tool arguments. Model size and coding ability alone are not enough. The provider abstraction made switching between cloud and local models straightforward, while the shared tool schemas exposed differences at the exact boundary that matters to this system.

A follow-up benchmark should run the same bounded coding task (same repository snapshot, prompt, tool permissions, and test command) on each provider multiple times. Record task completion, tests passed, invalid/failed tool calls, number of model turns, wall-clock time, and token use. The small Qwen model should be replaced with a tool-capable local model before treating it as a fair code-quality comparison.

Advanced RAG technique and evaluation

The project uses **cross-encoder reranking**, an advanced retrieval technique. Dense vector search finds a candidate set efficiently; a cross-encoder then reads each query and candidate passage together and reorders those candidates by relevance. This second stage targets the common case where semantically related documents are retrieved but the best answer page is not ranked first.

Evaluation setup

The checked-in evaluator runs 10 fixed RAGAs documentation questions with known expected source-page substrings. It compares plain vector search with vector search plus reranking. It reports top-1 (expected page ranked first) and hit@3 (expected page appears anywhere among the top three). The evaluation was re-run against the project index for this report.

Retrieval mode	Top-1 correct	Hit@3	Interpretation
Vector search only	7 / 10 (70%)	9 / 10 (90%)	Strong top-three coverage, but three queries did not put the expected page first.
Vector + cross-encoder rerank	9 / 10 (90%)	9 / 10 (90%)	Two additional expected pages move to rank one; top-three coverage is unchanged.

What the results suggest

On this small, fixed set, reranking improves the rank of the most relevant passage: top-1 rises by 20 percentage points (7/10 to 9/10). Hit@3 stays at 9/10, so the observed gain is primarily better ordering, not broader candidate recall. This matches the pipeline design: the cross-encoder reorders the vector candidate set and cannot recover a relevant page that vector retrieval failed to include.

The remaining miss matters: for the LangChain integration query, reranking fails to return the expected source within the top three; this points to candidate recall, query wording, or ranking quality as areas for investigation. The synthetic test-set-generation query improves from a vector miss to a reranked hit.

Limitations

Ten questions are a useful smoke test, not a statistically strong benchmark. Questions and expected pages are hand-authored; there are no repeated trials, confidence intervals, latency/cost comparisons, or human judgments of passage usefulness. These results measure source ranking only, not answer factuality or downstream coding-task success. Reranker performance also depends on the fixed embedding candidate pool (top 20).

Practical insight

Reranking is a good fit when first-result precision matters and a modest local scoring step is acceptable. To improve recall, first inspect whether expected pages enter the vector candidate pool; consider query reformulation or a larger candidate set only if that diagnosis supports the added cost.

Reflection: what worked and what to improve

What worked

The agent loop is provider-independent, and its explicit iteration limit prevents unbounded tool cycles. The confirmation default keeps file writes, edits, and shell commands visible to the user. The MCP client and server configuration make filesystem, Context7, and custom RAG tools available through one agent interface. RAG ingestion is persistent and separate from query-time use, and the reranking evaluation gives a concrete before/after measure rather than relying only on anecdotal answers.

What did not work as well

The demo notes describe integration friction from changes in MCP SDK naming and from coordinating a synchronous agent loop with an asynchronous MCP client. They also record inconsistent provider behavior: malformed cloud tool arguments on some long responses, a local model that did not emit structured tool calls, and Groq rate limits. MCP filesystem tools require absolute paths, so the working directory had to be made explicit in the system prompt. The 1.5B local model was constrained by available hardware.

What I would improve next

• **Run a reproducible paired LLM benchmark.** Freeze one coding task and repo snapshot; execute it for both providers under identical tool permissions; preserve transcripts and objective test results; report retries, steps, latency, and tokens.

• **Improve local-model suitability.** Select a local model that supports structured tool calling and fits available hardware; keep the provider interface and task constant when comparing it with Groq.

• **Broaden RAG evaluation.** Add more questions, independently review expected sources, track whether the expected page appears in the initial candidate pool, and measure latency as well as rank metrics.

• **Test failure paths end-to-end.** Exercise malformed arguments, denied tool requests, unavailable MCP services, rate limiting, and max-iteration stopping with repeatable tests and captured outputs.

Evidence consulted

The report is based on the checked-in project README, SPEC.md, PLAN.md, Project_Plan.md, docs/DEMO_SCRIPT.md, the provider/agent/MCP/RAG implementation, and the fresh run of servers.rag_server.evaluate. The original image is genie_code_architecture.png. The model comparison is intentionally limited to the qualitative observations recorded in the demo notes; no unrecorded experiment is represented as fact.

Bottom line: CodeMax integrates a modular coding-agent loop with three MCP servers and a reranked documentation RAG pipeline. The retrieval experiment shows a measurable top-1 improvement; the next most important evidence gap is a controlled, identical-task comparison of two tool-capable LLMs.